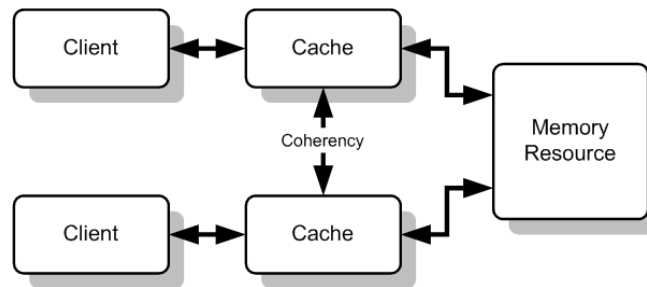


Lecture 8 — Cache Coherency

Patrick Lam & Jeff Zarnett

2026-09-27

Cache Coherency



– Careless hx, CC BY-SA 4.0, via Wikimedia Commons

Today we'll look at what support the architecture provides for memory ordering, in particular in the form of cache coherence. We'll be talking about cache coherence strategies that work for CPUs, where we don't get much choice. But what we're going to talk about works equally well for something like `redis` (`redict`) in a situation where we have a distributed cache in software. In a software scenario we might get to choose the configuration that we want; when it comes to the CPU we get whatever the hardware designers have provided to us.

The problem is, of course, that each CPU likely has its own cache(s). If it does, then the data in these may be out of sync – the value that CPU 1 has for a particular piece of data might be different from the value that CPU 4 has. The simplest method, and a horrible solution, would be the ability to declare some read/write variables as being non-cacheable (is that a word? Uncacheable?...). The compiler and OS and such will require the data to be read from main memory, always. This will obviously result in lower cache hit ratios, increased bus traffic, and terrible, terrible performance. Let's avoid that. What we want instead is *coherency*.

Cache coherency means that (1) the values in all caches are consistent; and (2) to some extent, the system behaves as if all CPUs are using shared memory.

In modern CPUs with three or four levels of cache, we frequently find that the level 3 cache isn't much faster than going to main memory. But this level is where the cache coherency communication can take place. This can be by making the cache shared between the different CPUs. And the L4 cache is frequently used for sharing data with the integrated graphics hardware on CPUs that have this feature. But for the most part we will imagine that caches are not shared, and we have to figure out how to get coherency between them. This is the case with an L1/L2 cache in a typical modern CPU as they are unique to the given core (i.e., not shared).

Cache Coherence Example. We will use this example to illustrate different cache coherence algorithms and how they handle the same situation.

Initially in main memory: $x = 7$.

1. CPU1 reads x , puts the value in its cache.
2. CPU3 reads x , puts the value in its cache.
3. CPU3 modifies $x := 42$
4. CPU1 reads $x \dots$ from its cache?

5. CPU2 reads x. Which value does it get?

Unless we do something, CPU1 is going to read invalid data.

Outside of a computing context, imagine you and several co-workers have some shared information, such as a meeting (in a specific room) in a shared online calendar (the one for the room). You (or anyone else) could make changes to this event. As mind-reading does not work, there are two ways that another invitee can know that something has changed: (1) they can check to see if anything has changed, or (2) they can be notified that a change has occurred.

The notification may contain the updated information in its entirety, such as “Event title changed to ‘Discuss User Permissions and Roles’”, or it may just tell you “something has changed; please check”. In transportation, you can experience both... in the same day. I [JZ] was flying to Frankfurt and going to catch a train. Air Canada sent me an e-mail that said “Departure time revised to 22:00” (20 minute delay); when I landed the Deutsche Bahn (German railways) sent me an e-mail that said “Something on your trip has changed; please check and see what it is in the app”... it was my train being cancelled. I don’t know why they couldn’t have e-mailed me that in the first place! It’s not like I was any less annoyed by finding out after taking a second step of opening an app.

Regardless of which method is used, we have to pick one. Otherwise, we won’t get the right answers.

Snoopy Caches. The simplest strategy is Snoopy caches [KMRS88]. No, not this kind of Snoopy (sadly):



It’s called Snoopy because the caches are, in a way, spying on each other: they are observing what the other ones are doing. This way, they are kept up to date on what’s happening and they know whether they need to do anything. They do not rely on being notified explicitly. This is a bit different from the transportation analogy, of course, but workable in a computer with a shared bus.

This is a distributed approach; no centralized state is maintained. Each cache with a copy of data from a block of main memory knows whether it is shared or not. All the CPUs are connected to a shared bus, and each CPU has its own cache controller. Whenever a CPU issues a memory write, the other CPUs are watching (colloquially, “snooping around”) to observe if that memory location is in their cache. If so, the CPU will need to take action.

What does action mean? In the flight plus train example, both kinds of action occurred. The Air Canada action was *update* – the information about the flight departure time was changed from 21:40 to 22:00 and at the time of becoming aware of the change, I got the new value immediately. The Deutsche Bahn action was *invalidate* – the information about the train was changed, but I didn’t know what had changed. All that I really knew is that the old information I had was out of date. When I needed that information again, I had to go get it myself from the source (their app). Either action (noting down the new, or knowing that what I have is out of date) is adequate for ensuring that I have the most up to date information. You may have a preference on which one you think is better, but unfortunately this is not your decision, neither as a user of the hardware of the computer nor as a person who wants to travel by plane or train.

Write-Through Caches

Let's put that into practice using write-through caches, the simplest type of cache coherence.

- All cache writes are done to main memory.
- All cache writes also appear on the bus.
- If another CPU snoops and sees it has the same location in its cache, it will either invalidate or update the data.

Invalidation is the most common protocol. It means the data in the cache of other CPUs is not updated, it's just noted as being out of date (invalid). In some cases, when you write to an invalidated location, you bypass the cache and go directly to memory (aka **write no-allocate**). This kind of thing happens if you're just doing $x = 42$; – it doesn't matter what value of x was there before; you're just overwriting it.

If we want to do a read and there's a miss, we can ask around the other caches to see who has the most recent cached version. This is a bit like going into a room and yelling "Does anybody have block...?", in some sort of multicast version of the card game "Go Fish". Regardless, the most recent value appears in memory, always, so if nobody else has it in cache (or they don't feel like sharing) you can get it from there.

There are also write broadcast protocols, in which case all versions in all caches get updated when there is a write to a shared block. But it uses lots of bandwidth and is not necessarily a good idea. It does, however, prevent the costly cache miss that follows an invalidate. Sadly, as we are mere users and not hardware architects, we don't get to decide which is better; we just have to live with whichever one is on the hardware we get to use. Bummer.

Write-Through Protocol. The protocol for implementing such caches looks like this. There are two possible states, **valid** and **invalid**, for each cached memory location. Events are either from a processor (**Pr**) or the **Bus**. Actions will be either a **Rd** (read) or **Wr** (write). We then implement the following state machine.

State	Observed	Generated	Next State
Valid	PrRd		Valid
Valid	PrWr	BusWr	Valid
Valid	BusWr		Invalid
Invalid	PrWr	BusWr	Valid
Invalid	PrRd	BusRd	Valid

There are write-allocate and write-no-allocate variants of write-through. In all cases, the written value goes to memory.

- write-allocate: Invalid/PrWr line has next state Valid: the written value is also immediately cached.
- write-no-allocate: Invalid/PrWr line has next state Invalid: the written value is not immediately cached.

Example. For simplicity (this isn't an architecture course), assume all cache reads/writes are atomic.¹ Using the same example as before:

Initially in main memory: $x = 7$.

1. CPU1 reads x , puts the value in its cache. (valid)
2. CPU3 reads x , puts the value in its cache. (valid)

¹If you're a hardware person, this line probably makes you cry. There's a whole lot that goes into making this work. There are potential write races, which have to be dealt with by contending for the bus and then completing the transaction, possibly restarting a command if necessary. If we have a split transaction bus it's really ugly, because we can have multiple interleaved misses. And down the rabbit hole we go.

3. CPU3 modifies $x := 42$. (write to memory)
 - CPU1 snoops and marks data as invalid.
4. CPU1 reads x , from main memory. (valid)
5. CPU2 reads x , from main memory. (valid)

Write-Back Caches

Let's try to improve performance. What if, in our example, CPU3 writes to x 3 times in rapid succession? It's unpleasant to have to flush that to memory three times when we could do it only once. Let's try to delay the write to memory as long as possible. At minimum, we need support in hardware for a "dirty" bit, which indicates the our data has been changed but not yet been written to memory.

Write-Back Implementation. The simplest type of write-back protocol (MSI) uses 3 states instead of 2:

- **Modified** – only this cache has a valid copy; main memory is **out-of-date**.
- **Shared** – location is unmodified, up-to-date with main memory; may be present in other caches (also up-to-date).
- **Invalid** – same as before.

The initial state for a memory location, upon its first read, is "shared". The implementation will only write the data to memory if another processor requests it. During write-back, a processor may read the data from the bus.

MSI Protocol. Here, bus write-back (or flush) is **BusWB**. Exclusive read on the bus is **BusRdX**.

State	Observed	Generated	Next State
Modified	PrRd		Modified
Modified	PrWr		Modified
Modified	BusRd	BusWB	Shared
Modified	BusRdX	BusWB	Invalid
Shared	PrRd		Shared
Shared	BusRd		Shared
Shared	BusRdX		Invalid
Shared	PrWr	BusRdX	Modified
Invalid	PrRd	BusRd	Shared
Invalid	PrWr	BusRdX	Modified

MSI Example. Using the same example as before:

Initially in main memory: $x = 7$.

1. CPU1 reads x from memory. (BusRd, shared)
2. CPU3 reads x from memory. (BusRd, shared)
3. CPU3 modifies $x = 42$: (PrWr, shared)
 - Generates a BusRdX.
 - CPU1 snoops and invalidates x .
4. CPU1 reads x : (PrRd, invalid)
 - Generates a BusRd.
 - CPU3 writes back the data and sets x to shared.
 - CPU1 reads the new value from the bus as shared.
5. CPU2 reads x from memory. (BusRd, shared)

An Extension to MSI: MESI

The most common protocol for cache coherence is MESI. This protocol adds yet another state:

- **Modified** – only this cache has a valid copy; main memory is **out-of-date**.
- **Exclusive** – only this cache has a valid copy; main memory is **up-to-date**.
- **Shared** – same as before.
- **Invalid** – same as before.

MESI allows a processor to modify data exclusive to it, without having to communicate with the bus. MESI is safe. The key is that if memory is in the E state, no other processor has the data. The transition from E to M does not have to be reported over the bus, which potentially saves some work and reduces bus usage.

MESIF: Even More States! MESIF (used in the latest i7 processors): **Forward** – basically a shared state; but, current cache is the only one that will respond to a request to transfer the data.

Hence: a processor requesting data that is already shared or exclusive will only get one response transferring the data. Under a simpler MESI scheme you could get multiple caches trying to answer, which leads to bus arbitration or contention. The existence of an F state permits more efficient usage of the bus.

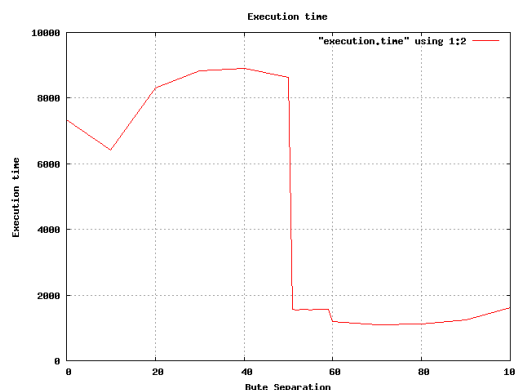
False Sharing

False sharing is something that happens when our program has two unrelated data elements that are mapped to the same cache line/location. That can be because of bad luck (hash collision kind of problem), but it often takes place because the data elements are stored consecutively. Let's consider an example from [Men08]:

```
char a[10];  
char b[10];
```

These don't overlap, but are almost certainly allocated next to each other in memory. If a thread is writing to `a` and they share a cache line or block, then `b` will be invalidated; any CPU working on `b` will be forced to fetch the newest value from memory. This can be avoided by forcing some separation between these two arrays. One way would be to heap allocate both arrays. Usually if you do this you will find that they are not both located at the same location (but it's not guaranteed). So the other alternative is to make both arrays bigger than they need to be such that we're sure they don't overlap.

Consider the graph below that shows what happens in a sample program reading and writing these two arrays, as you increase the size of arrays `a` and `b` (noting that byte separation of 11 means they are adjacent; anything less than that and they overlap). This does waste space, but is it worth it?



Execution time graph showing $5\times$ speedup by “wasting” some space [Men08].

At separation size 51 there is a huge drop in execution time because now we are certainly putting the two arrays in two locations that do not have the false sharing problem. Is wasting a little space worth it? Yes! Also – putting these arrays in a struct and padding the struct can also help with enabling future updates to the struct.

Software Implementation

A previous exam question on caching asked students to write a pseudocode description of behaviour for a distributed software cache that uses the MESI states and has write-back behaviour. This cache is for data items retrieved from a database, so if the item is not in any node's cache, write down retrieve item *i* from database.

You can assume the cache to be of significant size and while the question said you can assume that the least-recently-used (LRU) algorithm is used for replacement, you don't really have to consider replacement in this situation at all. As a practice problem for consideration, think about what modification(s) you would need to make for that scenario.

As the cache is distributed, we do need to consider what happens if a node comes online and joins the cluster, and what happens if a node is going to shut down and leave the cluster. You may ignore situations like crashes or network outages and you can assume all sent messages are reliably delivered/received.

```

Current Node Shutdown {
    for i in items {
        if i is in state M {
            write i to the database
        }
    }
    for node n in known_nodes {
        send leaving message to n
    }
}

Node Leaves ( node n ) {
    Remove n from known_nodes
}

Get Item ( item i ) {
    if i is in local cache {
        return i
    } else {
        for node n in known_nodes {
            if n has item i
                add to cache ( i )
                set i state to S
                return i
            }
        }
        retrieve i from database
        add to cache ( i )
        set i state to E
        return i
    }
}

Other Node Searching ( item i ) {
    if i is in local cache {
        if i is in state M {
            write i to the database
            set i state to S
            return i
        } else if i is in state E {
            set i state to S
            return i
        } else if i is in state S {
            return i
        }
    }
    return null //0r other indicator of not-found
}

Update Item ( item i ) {
    if i is in local cache {
        if i is in state S {
            for node n in known_nodes {
                send invalidate i message to n
            }
            set i state to M
        } else if i is in state E {
            set i state to M
            return
        } else if i is in state M {
            return // Nothing to do
        }
    }
    add i to the cache in state M
}

Invalidate ( item i ) {
    if i is in local cache {
        set i state to I
    }
}

```

References

- [KMRS88] Anna R. Karlin, Mark S. Manasse, Larry Rudolph, and Daniel D. Sleator. Competitive snoopy caching. *Algorithmica*, 3(1-4):79–119, 1988. URL: <http://dx.doi.org/10.1007/BF01762111>.
- [Men08] Gaetano Mendola. False sharing hits again!, 2008. Online; accessed 7-December-2018. URL: <http://cpp-today.blogspot.com/2008/05/false-sharing-hits-again.html>.